

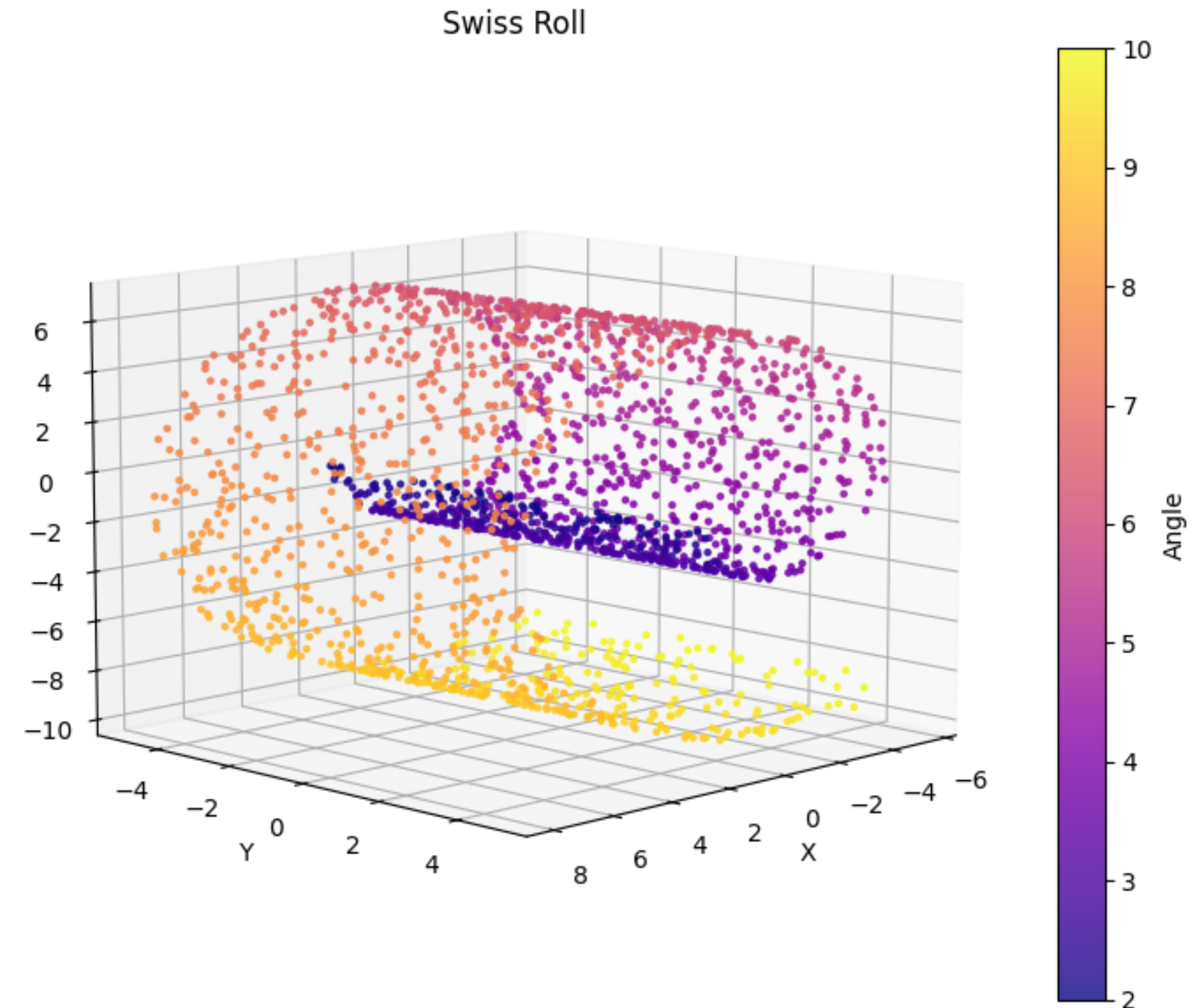
Manifold Sculpting

Leonardo Angellotti

Swiss Roll problem

$$x(t) = t \sin(t), \quad y(t) \sim \mathcal{U}(-5,5), \quad z(t) = t \cos(t).$$

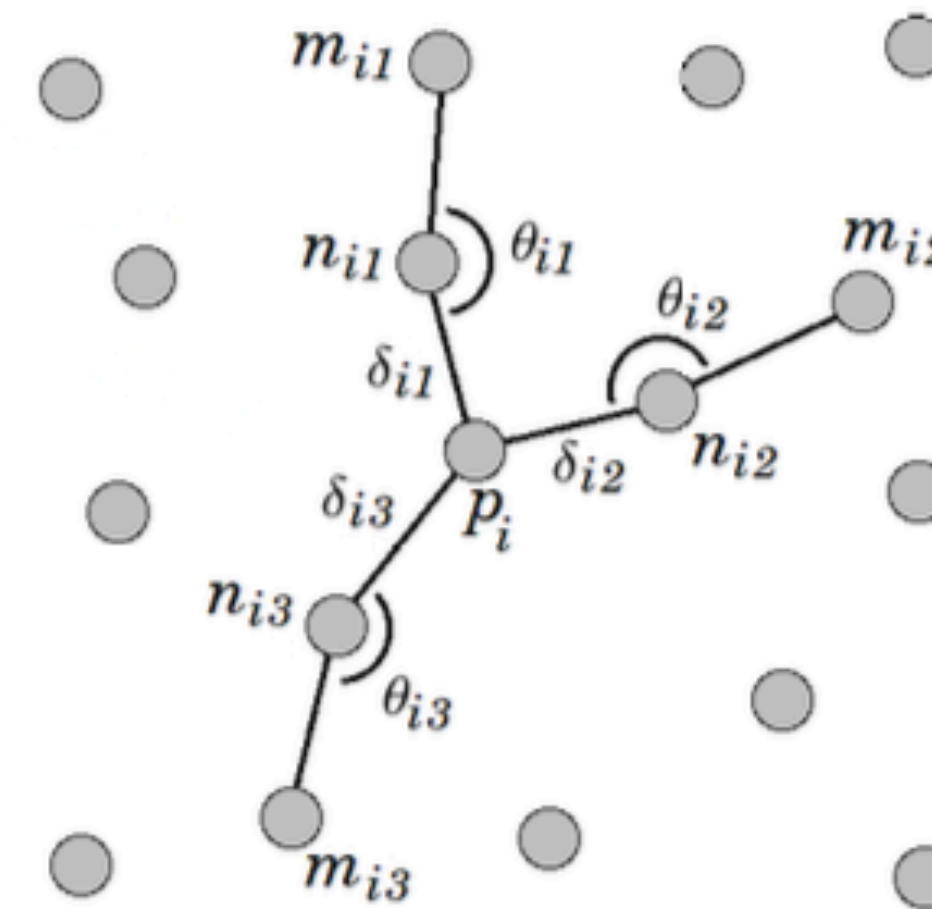
The angle t is stored separately for colouring purposes.



Algorithm

-
- 1 Find the k nearest neighbors of each point.
 - 2 Compute a set of relationships between neighbors.
 - 3 Optionally align axes with principle components.
 - 4 While the stopping criteria has not been met...
 - a. Scale the data in the non-preserved dimensions.
 - b. Adjust points to restore the relationships.
 - 5 Project the data.
-

(a) A high-level overview of Manifold Sculpting.



(b) Local relationships

EmbeddingRefiner: Manifold Sculpting Algorithm

iteratively constructs a low-dimensional embedding while preserving local neighbourhood structure.

1. Neighbourhood graph:

For each point i , find its k nearest neighbours. Denote their indices by $\mathcal{N}_i$ and distances by $\delta_{i,j}$

2. Angular targets:

For each neighbour $j \in \mathcal{N}_i$, choose a third point $k \in \mathcal{N}_j$ such that the angle

$\theta_{i,j,k}$ between vectors $\mathbf{a} = \mathbf{x}_i - \mathbf{x}_j$ and $\mathbf{b} = \mathbf{x}_k - \mathbf{x}_j$ is closest to π .

Store this angle as $\theta_{i,j}^0$

3. Initialisation:

Align the data to its principal components via PCA. Define two sets of coordinate axes:

preserved dimensions $\mathcal{D}_{\text{pres}}$ and shrinking dimensions $\mathcal{D}_{\text{shrink}}$

Point adjustment:

Perform a breadth-first traversal of the neighbourhood graph starting from a random point.

For each visited point p , adjust its preserved coordinates $y_{p,d}$ by small increments $\pm\eta$ to minimise the error

$$E_p = \sum_{j \in \mathcal{N}_p} w_{p,j} \left[\frac{1}{2} \left(\frac{|\mathbf{y}_p - \mathbf{y}_j| - \delta_{p,j}}{\bar{\delta}} \right)^2 + \left(\frac{\theta_{p,j} - \theta_{p,j}^0}{\pi} \right)^2 \right]$$

where $w_{p,j}$ is a weight that emphasises neighbours already adjusted in this epoch.

4. **Scaling:**

Multiply coordinates in $\mathcal{D}_{\text{shrink}}$ by a factor $\sigma < 1$ (“damping”) and, if necessary, divide coordinates in $\mathcal{D}_{\text{pres}}$ by σ to maintain the average neighbour distance

$$\bar{\delta} = \frac{1}{|E|} \sum_{(i,j) \in E} |\mathbf{y}_i - \mathbf{y}_j|$$

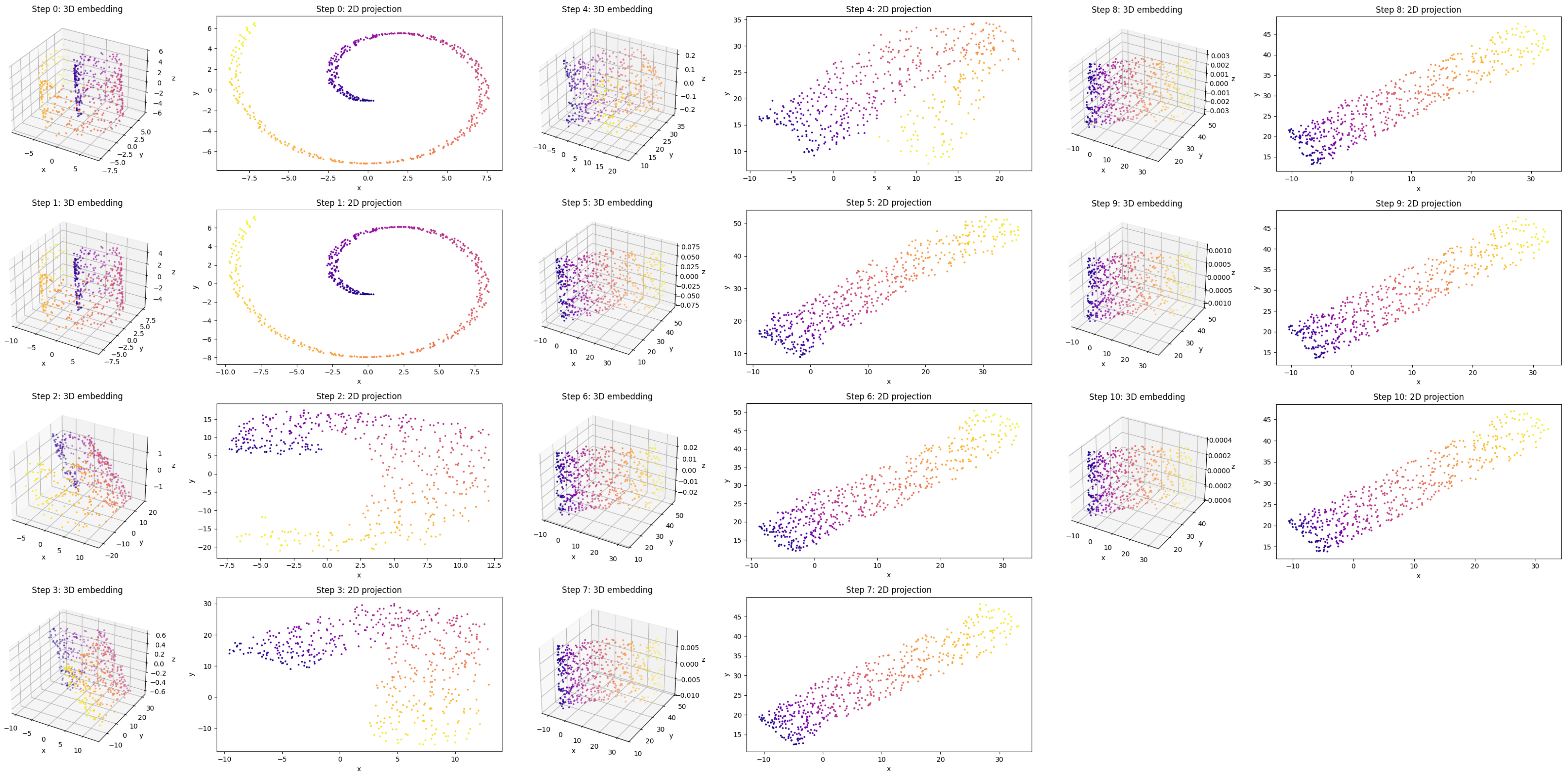
where E is the set of all directed edges in the neighbourhood graph.

Learning rate adaptation:

Increase the learning rate if many coordinate updates succeed; otherwise decrease it.
Repeat until convergence or a maximum number of epochs is reached.

This procedure converges to an embedding where local distances and angles approximate those of the original data, effectively "sculpting" away undesired dimensions.

Unrolling process



Isomap

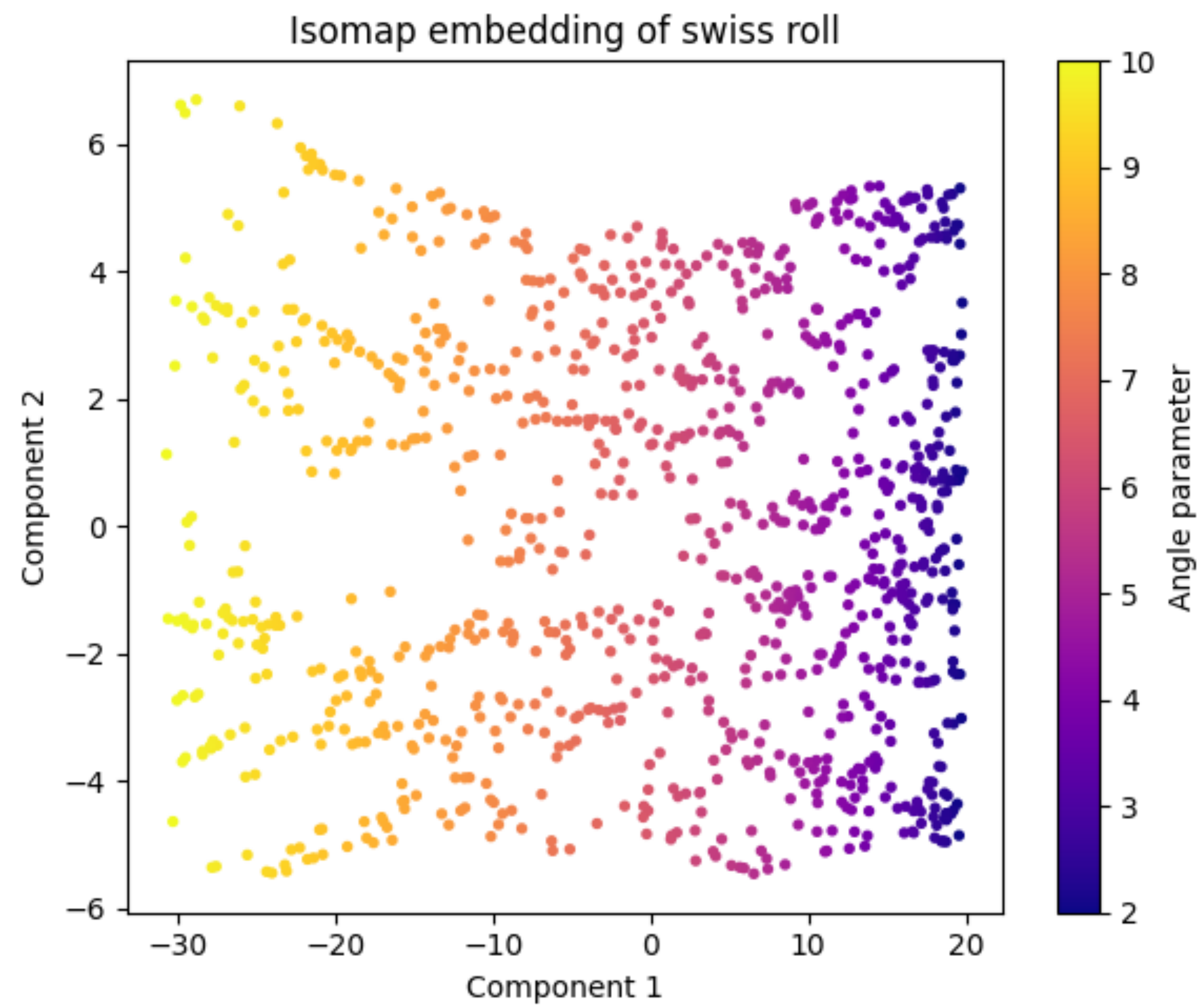
Isomap extends Multidimensional Scaling by replacing Euclidean distances with geodesic distances along the data manifold. To compute these distances we:

1. Build a k -nearest neighbour graph G on the data points.
2. Assign edge weights equal to the Euclidean distances between connected vertices.
3. Compute the shortest-path distances $d_{i,j}$ in G using the Floyd–Warshall algorithm or Dijkstra's algorithm.

After obtaining the geodesic distance matrix D , we perform **double centring** to form the Gram matrix

$$B = -\frac{1}{2} \left(I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top \right) D^2 \left(I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top \right)$$

whose eigenvectors corresponding to the largest eigenvalues provide the low-dimensional embedding.



Kernel PCA generalises PCA by implicitly mapping data to a high-dimensional feature space via a kernel function $k(\mathbf{x}, \mathbf{y})$.

We form the kernel matrix K with entries $K_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j)$, centre it, and then perform an eigen decomposition. The embeddings are obtained by scaling the eigenvectors by the square roots of their eigenvalues.

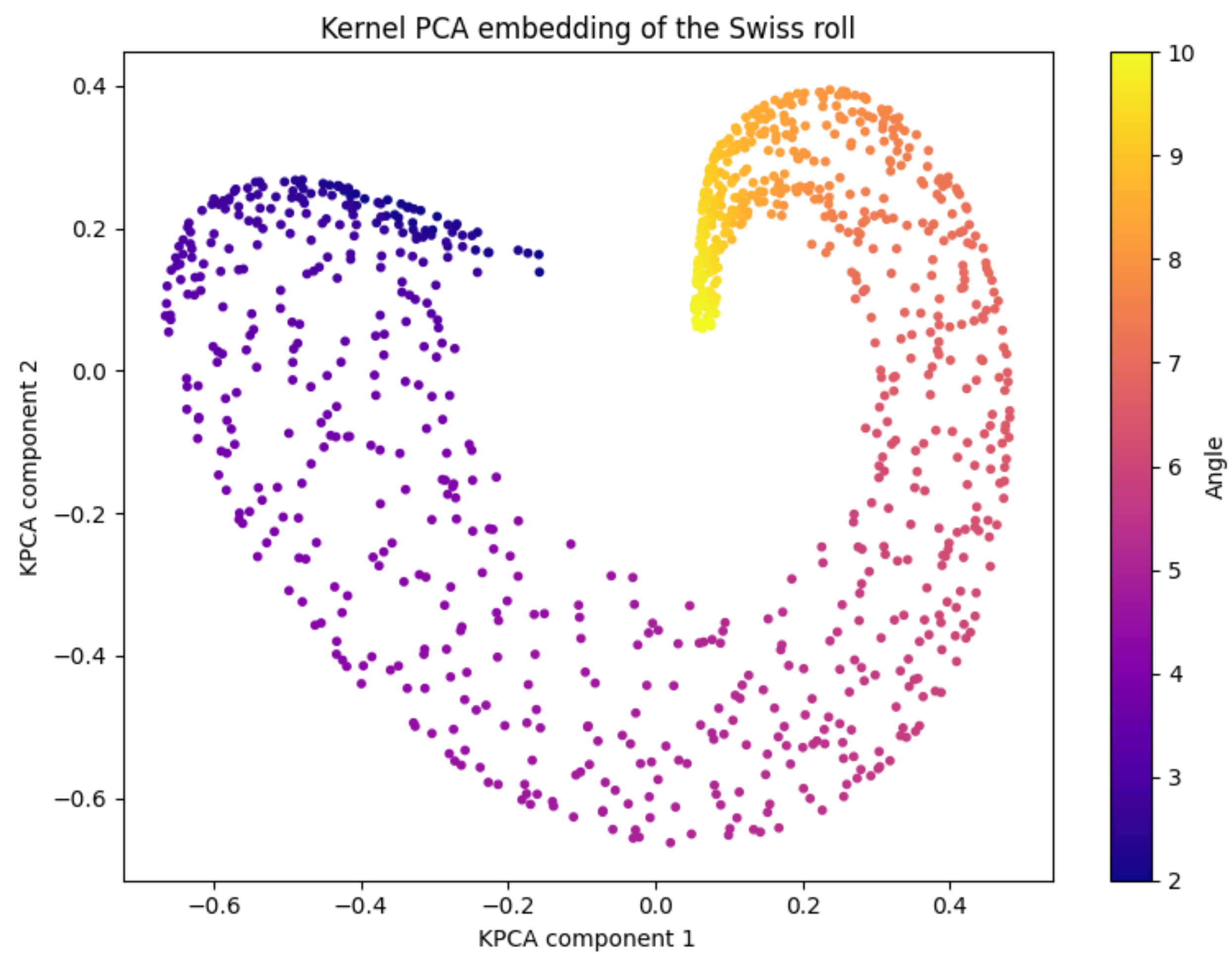
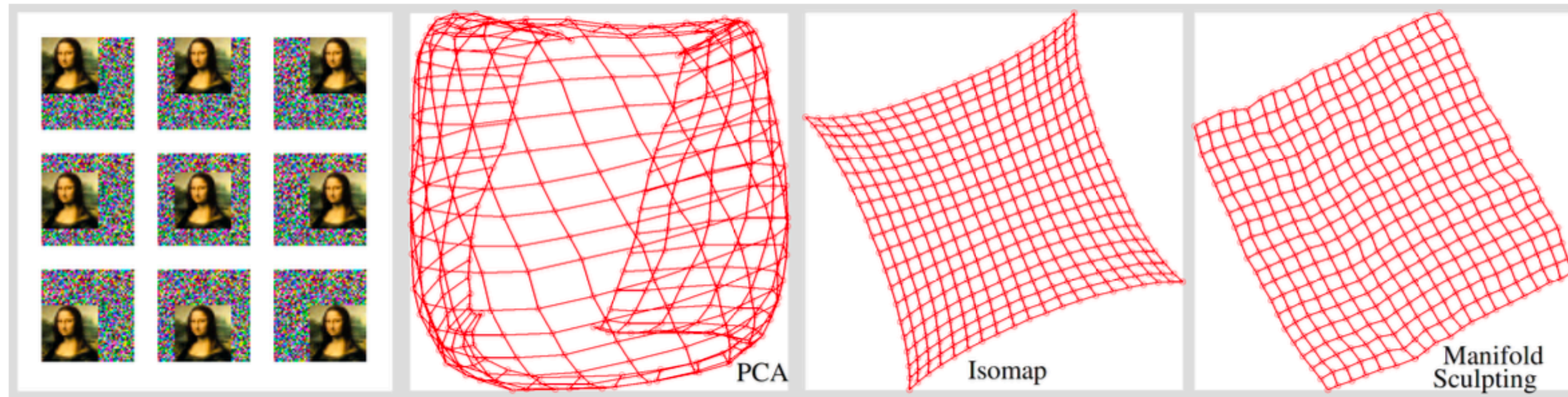


Image experiment



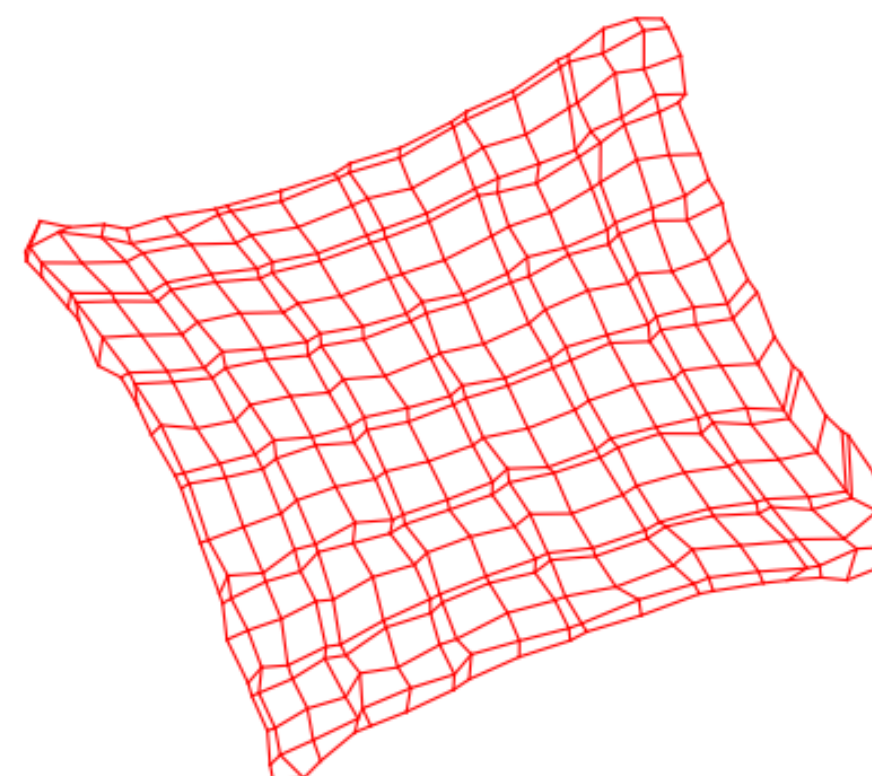


Recovered 2D Manifolds from Translated Face Images

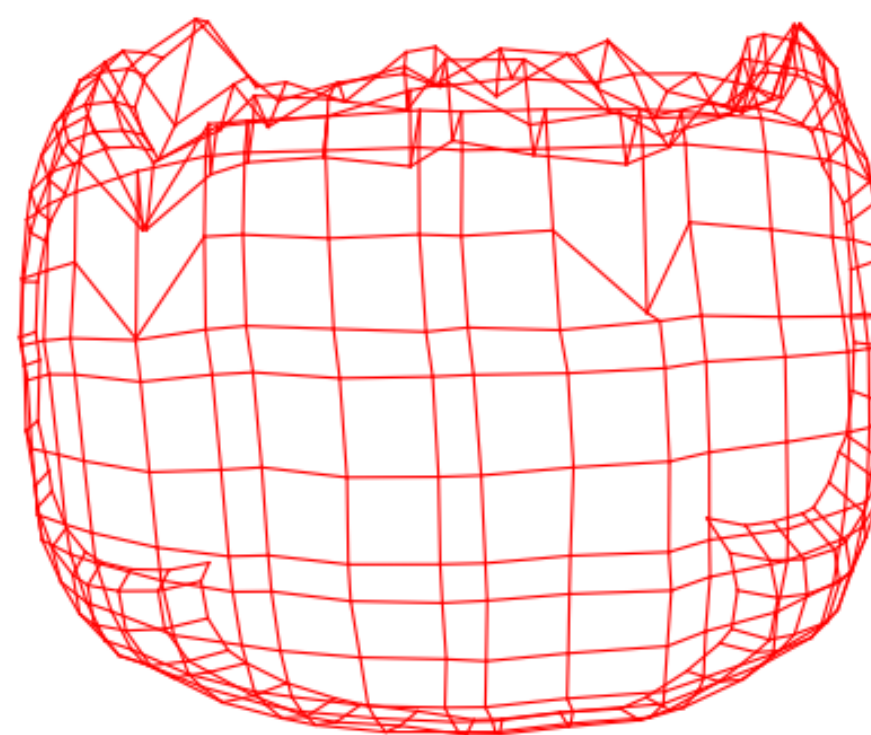
PCA



Isomap

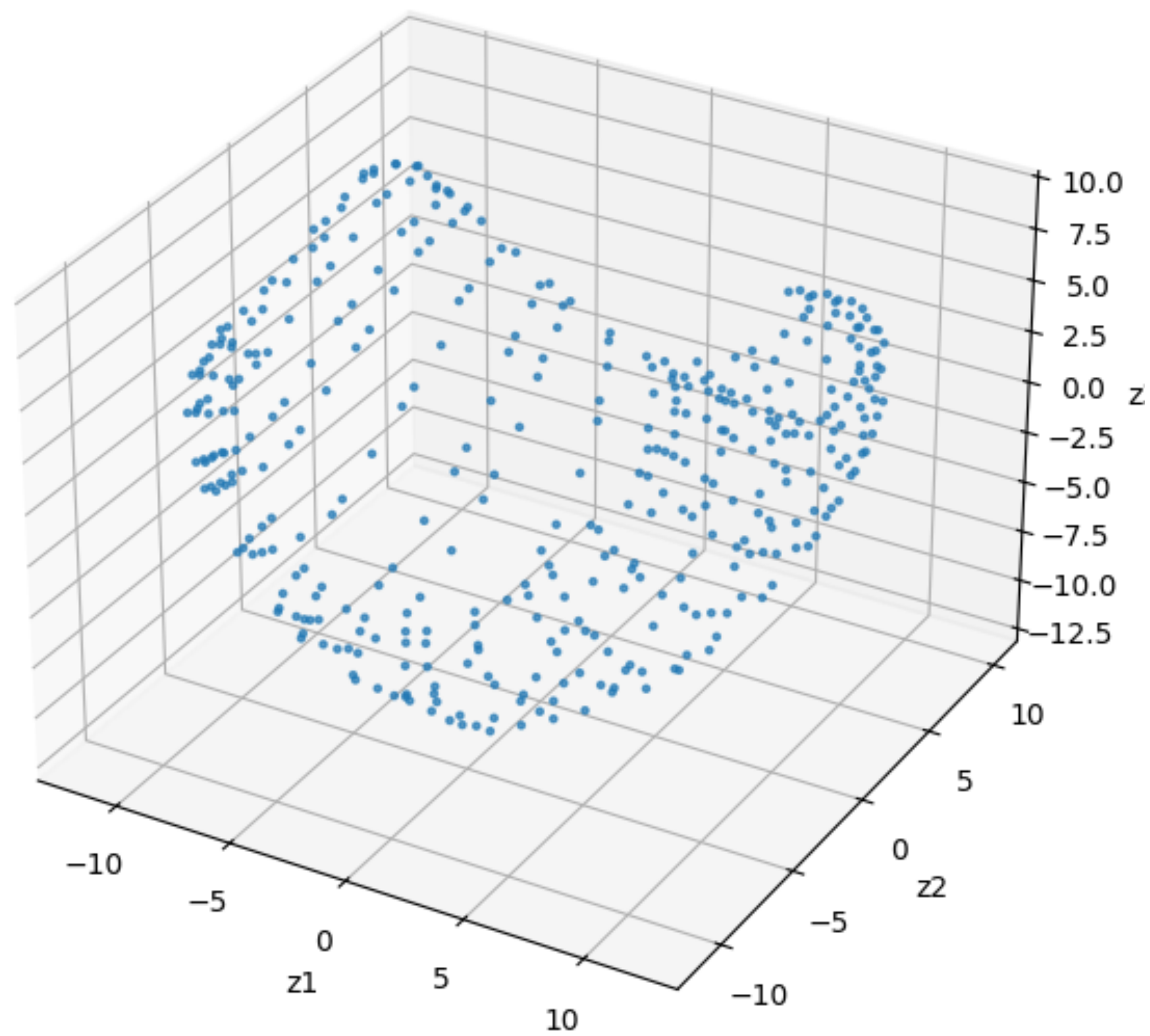


Manifold Sculpting



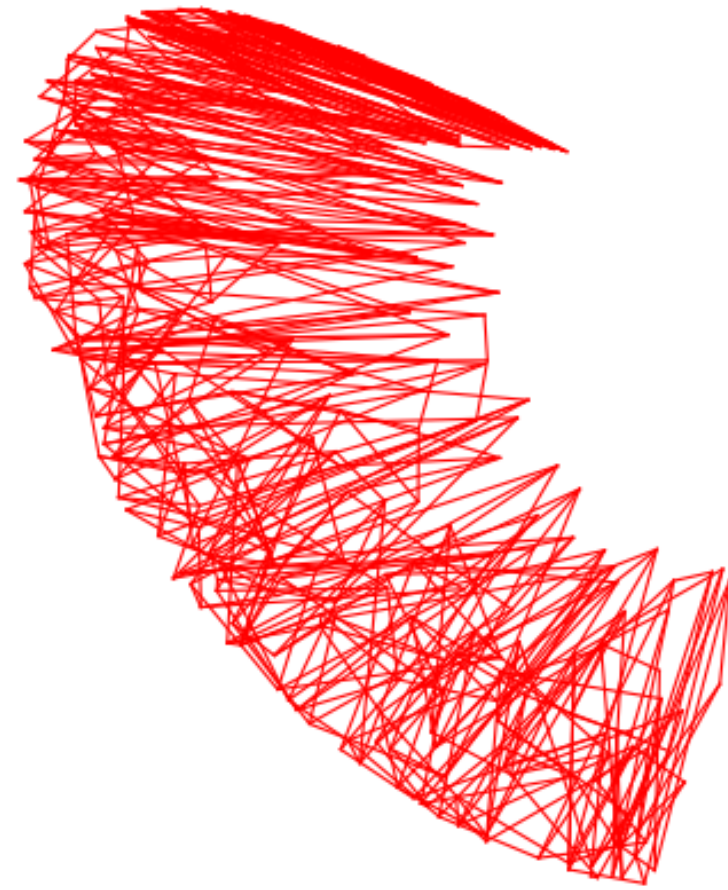
Let's cheat on the results

3D latent representation of images

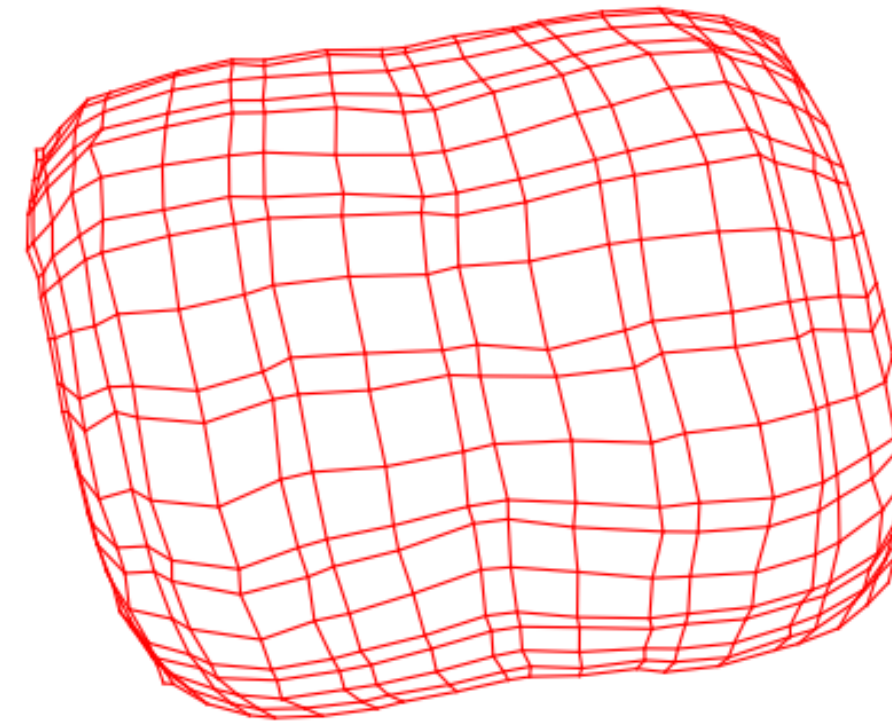


Recovered 2D Manifolds from Translated Face Images

PCA



Isomap



Manifold Sculpting

